Tarea LL1

Autor:
Karla Mariana Duque Callejas

Asignatura:
Lenguajes de Programación y Transducción

Grupo: G02

Fecha: 11 de abril de 2026

Índice

1. Ejercicio 1 **2**
- 1.1. Descripción del problema . . . 2
- 1.2. Solución primeros cinco items . . . 2
  - 1.2.1. Eliminar recursión por izquierda . . . 2
  - 1.2.2. Gramática corregida: . . . 3
  - 1.2.3. Conjunto de PRIMEROS y SIGUIENTES . . . 3
  - 1.2.4. Conjunto PREDICCION y LL(1) . . . 3
  - 1.2.5. Condiciones para LL(1) . . . 4
- 1.3. Función para cada no terminal en un ASDR . . . 5

2. Ejercicio 2 **6**
- 2.1. Descripción del problema . . . 6
- 2.2. Solución primeros cuatro items . . . 7
  - 2.2.1. Conjunto de PRIMEROS y de de SIGUIENTES . . . 7
  - 2.2.2. Conjunto de PREDICCION y LL(1) . . . 7
  - 2.2.3. Condiciones para LL(1) . . . 8
- 2.3. Función para cada no terminal en un ASDR . . . 9
  - 2.3.1. Eliminar recursión por izquierda . . . 9
  - 2.3.2. Gramática con recursividad directa por izquierda . . . 9
  - 2.3.3. Gramática sin recursividad por izquierda . . . 10
  - 2.3.4. Implementación del ASDR . . . 10

3. Ejercicio 3 **12**
- 3.1. Descripción del problema . . . 12
- 3.2. Solución primeros cinco items . . . 13
  - 3.2.1. Eliminar recursión por izquierda . . . 13
  - 3.2.2. Gramática corregida . . . 13
  - 3.2.3. Conjunto de PRIMEROS y SIGUIENTES . . . 13
  - 3.2.4. Conjunto PREDICCION y LL(1) . . . 14
  - 3.2.5. Condiciones para LL(1) . . . 15
- 3.3. Función para cada no terminal en un ASDR . . . 15

1. Ejercicio 1

1.1. Descripción del problema

Eliminar recursión por izquierda , hallar conjunto de PRIMEROS, SIGUIENTES, PREDICCION, determinar si la gramática es LL(1) e implementar la función para cada no terminal en un ASDR.

Gramática:

S → A B C
S → D E
A → **dos** B **tres**
A → ε
B → B **cuatro** C **cinco**
B → ε
C → **seis** A B
C → ε
D → **uno** A E
D → B
E → **tres**

Figura 1: Gramática 1

1.2. Solución primeros cinco items

1.2.1. Eliminar recursión por izquierda

Se transforman las reglas de tipo $A \to A\ \alpha \mid \beta$ (donde β no empieza con A) en reglas recursivas por derecha: $A \to \beta\ A'$ y $A' \to \alpha\ A' \mid \epsilon$.

1. Identificar reglas problemáticas: $B \to B\ cuatro\ C\ cinco \mid \epsilon$
2. Identificar $A,\ \alpha$ y β: $A = B,\ \alpha = cuatro\ C\ cinco\ \beta = \epsilon$
3. Reemplazando: $B \to \epsilon\ B'$ y $B' \to\ cuatro\ C\ cinco\ B' \mid \epsilon$.
4. Simplificando $B \to B'$ y $B' \to\ cuatro\ C\ cinco\ B' \mid \epsilon$.

1.2.2. Gramática corregida:

$$
\begin{aligned}
S &\rightarrow ABC \mid DE \\
A &\rightarrow \textbf{dos } B \textbf{ tres} \mid \epsilon \\
B &\rightarrow B' \\
B' &\rightarrow \textbf{cuatro } C \textbf{ cinco } B' \mid \epsilon \\
C &\rightarrow \textbf{seis } A\ B \mid \epsilon \\
D &\rightarrow \textbf{uno } A\ E \mid B \\
E &\rightarrow \textbf{tres}
\end{aligned}
$$

1.2.3. Conjunto de PRIMEROS y SIGUIENTES

Se hizo uso del código psp.py del repositorio `https://github.com/Mariana909/PRIMEROS_SIGUIENTES_PREDICCION.git` con Gramatica1.txt como archivo de entrada, este archivo se encuentra en el repositorio `https://github.com/Mariana909/LL1.git`.

No Terminal	PRIMEROS	SIGUIENTES
S	{ _, cuatro, dos, seis, tres, uno }	{ $ }
A	{ _, dos }	{ $, cinco, cuatro, seis, tres }
B	{ _, cuatro }	{ $, cinco, seis, tres }
B'	{ _, cuatro }	{ $, cinco, seis, tres }
C	{ _, seis }	{ $, cinco }
D	{ _, uno }	{ tres }
E	{ tres }	{ $, tres }

Figura 2: Conjunto de PRIMEROS y de SIGUIENTES de Gramatica1

1.2.4. Conjunto PREDICCION y LL(1)

Además de la tabla de PRIMEROS y SIGUIENTES, se obtuvo la tabla de PREDICCION a partir de la ejecución anterior.

Regla	Conjunto de Prediccion
S -> A B C	{ $, cuatro, dos, seis }
S -> D E	{ tres, uno }
A -> dos B tres	{ dos }
A -> _	{ $, cinco, cuatro, seis, tres }
B -> B'	{ $, cinco, cuatro, seis, tres }
B' -> cuatro C cinco B'	{ cuatro }
B' -> _	{ $, cinco, seis, tres }
C -> seis A B	{ seis }
C -> _	{ $, cinco }
D -> uno A E	{ uno }
D -> _	{ tres }
E -> tres	{ tres }

Figura 3: Conjunto de PREDICCION de Gramatica1

1.2.5. Condiciones para LL(1)

Se evalúa si la Gramatica1 cumple las condiciones para ser LL(1)

- **Sin recursividad por izquierda**: Se modificó la gramática para eliminar la recursividad por izquierda, por lo que se cumple esta condición.
- **Sin factores comunes por izquierda**: La gramática no cuenta con factores comunes por izquierda, por lo que se cumple esta condición.
- **Conjunto de PREDICCION**: Una gramática es LL(1) si, para cada no-terminal con dos o más reglas de producción, los conjuntos de predicción de dichas reglas son disjuntos, es decir, su intersección es el conjunto vacío. Revisando la Figura 3, se verifica que en todos los no-terminales de esta gramática (S, A,B,B',C,D,E) los conjuntos de predicción no comparten ningún símbolo. Por tanto, dado cualquier símbolo de entrada, el analizador puede determinar unívocamente qué producción aplicar sin necesidad de retroceso.

Se concluye que Gramatica1 es LL(1)

1.3. Función para cada no terminal en un ASDR

Se tomó como base el código `parser.py` del repositorio `https://github.com/Mariana909/ARBOL_SINTACTICO.git`, sobre el cual se realizaron las siguientes adaptaciones: se redefinieron los tokens para que correspondan a los terminales de cada gramática, se implementó una función por cada no terminal siguiendo la estructura de la gramática corregida, y se conservó el módulo de generación de imágenes que produce el árbol de sintaxis para cada cadena de entrada.

En el repositorio `https://github.com/Mariana909/LL1.git` se encuentra la implementación descrita, en el archivo `parser_gramatica1.py`. Para ejecutarla se usa el comando:

```
python3 parser_gramatica1.py entrada_g1.txt
```

donde `entrada_g1.txt` contiene las cadenas a analizar, una por línea. El programa imprime en consola si cada cadena es aceptada o rechazada, y genera una imagen del árbol de sintaxis por cada entrada.

Salidas:

```
ACEPTADA  "dos tres"
ACEPTADA  "dos cuatro cinco tres"
ACEPTADA  "seis"
RECHAZADA  "uno tres"
RECHAZADA  "cinco cuatro tres"
RECHAZADA  "cuatro cinco tres"
```

Figura 4: Salidas parser_gramatica1.py con entrada_g1.txt

A continuación un ejemplo de cadena aceptada en la Figura 5 y una cadena rechazada en la Figura 6

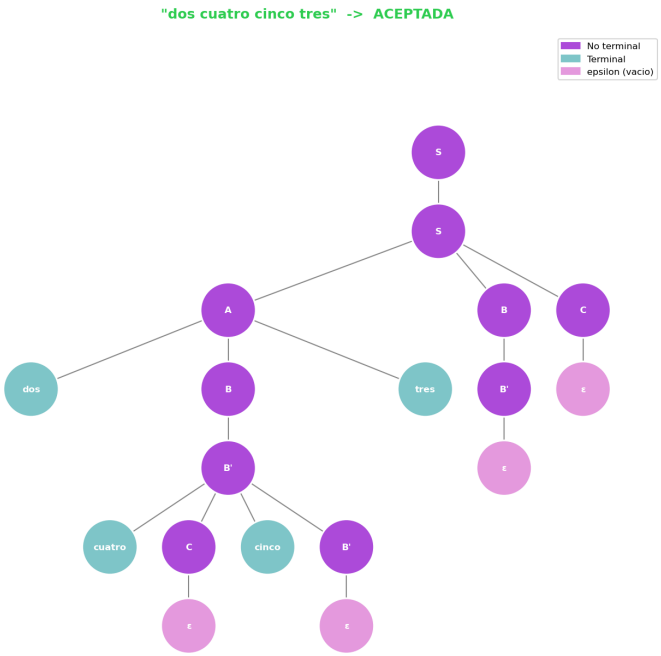

Figura 5: Árbol de sintaxis de la entrada `dos cuatro cinco tres`

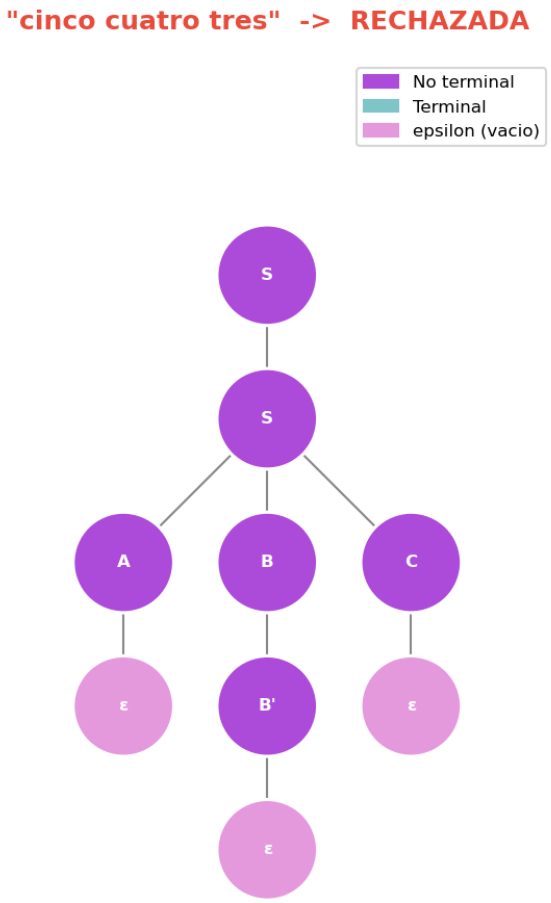

Figura 6: Árbol de sintaxis de la entrada `cinco cuatro tres`

2. Ejercicio 2

2.1. Descripción del problema

Hallar conjunto de PRIMEROS, SIGUIENTES, PREDICCION, determinar si la gramática es LL(1) e implementar la función para cada no terminal en un ASDR.

Gramática:

$S \rightarrow B$ **uno**
$S \rightarrow$ **dos** C
$S \rightarrow \varepsilon$
$A \rightarrow S$ **tres** $B\ C$
$A \rightarrow$ **cuatro**
$A \rightarrow \varepsilon$
$B \rightarrow A$ **cinco** C **seis**
$B \rightarrow \varepsilon$
$C \rightarrow$ **siete** B
$C \rightarrow \varepsilon$

Figura 7: Gramática 2

2.2. Solución primeros cuatro items

2.2.1. Conjunto de PRIMEROS y de de SIGUIENTES

Se hizo uso del código psp.py del repositorio `https://github.com/Mariana909/PRIMEROS_SIGUIENTES_PREDICCION.git` con Gramatica2.txt como archivo de entrada, este archivo se encuentra en el repositorio `https://github.com/Mariana909/LL1.git`.

No Terminal	PRIMEROS	SIGUIENTES
S	{ _, cinco, cuatro, dos, tres, uno }	{ $, tres }
A	{ _, cinco, cuatro, dos, tres, uno }	{ cinco }
B	{ _, cinco, cuatro, dos, tres, uno }	{ $, cinco, seis, siete, tres, uno }
C	{ _, siete }	{ $, cinco, seis, tres }

Figura 8: Conjunto de PRIMEROS y de SIGUIENTES de Gramatica2

2.2.2. Conjunto de PREDICCION y LL(1)

Además de la tabla de PRIMEROS y SIGUIENTES, se obtuvo la tabla de PREDICCION a partir de la ejecución anterior.

Regla	Conjunto de Prediccion
S -> B uno	{ cinco, cuatro, dos, tres, uno }
S -> dos C	{ dos }
S -> _	{ $, tres }
A -> S tres B C	{ cinco, cuatro, dos, tres, uno }
A -> cuatro	{ cuatro }
A -> _	{ cinco }
B -> A cinco C seis	{ cinco, cuatro, dos, tres, uno }
B -> _	{ $, cinco, seis, siete, tres, uno }
C -> siete B	{ siete }
C -> _	{ $, cinco, seis, tres }

Figura 9: Conjunto de PREDICCION de Gramatica2

2.2.3. Condiciones para LL(1)

Se evalúa si la Gramatica2 cumple las condiciones para ser LL(1)

- **Sin recursividad por izquierda**: Se dice que una gramática tiene recursividad indirecta por izquierda cuando existe una cadena de derivaciones de la forma $A \Rightarrow^+ A\,\alpha$, donde cada paso expande el símbolo más a la izquierda. En esta gramática se verifica el ciclo $S \Rightarrow B \ldots \Rightarrow A \ldots \Rightarrow S \ldots$, lo que constituye recursividad indirecta por izquierda. Las reglas que la originan son $S \to B\,\mathbf{uno}$, $B \to A\,\mathbf{cinco}\,C\,\mathbf{seis}$ y $A \to S\,\mathbf{tres}\,B\,C$, pues en cada una el primer símbolo de la parte derecha es un no-terminal que eventualmente deriva al no-terminal de la izquierda.

- **Sin factores comunes por izquierda**: La gramática no cuenta con factores comunes por izquierda, por lo que se cumple esta condición.

- **Conjunto de PREDICCION**: Una gramática es LL(1) si, para cada no-terminal con dos o más reglas de producción, los conjuntos de predicción de dichas reglas son disjuntos, es decir, su intersección es el conjunto vacío. Revisando la Figura 9, se tiene que para el no terminal B los conjuntos de predicción de sus reglas comparten los símbolos cinco, tres y uno, por esta razón la gramática no cumple esta condición.

Se concluye que Gramatica2 no es LL(1)

2.3. Función para cada no terminal en un ASDR

Antes de implementar la función se debe convertir la gramática a LL(1), por lo que se corrige la recursividad indirecta por izquierda.

2.3.1. Eliminar recursión por izquierda

Convertir la recursión indirecta a directa: para convertir recursividad indirecta en directa, hay que sustituir las producciones del no-terminal que aparece en primera posición, reemplazando esa aparición por todas sus alternativas. Se repite hasta que el mismo no-terminal quede en primera posición.

1. Reemplazar S dentro de $A \to S$ **tres** B C por todas sus producciones ($S \to B$ **uno**, $S \to$ **dos** C, $S \to \varepsilon$):

$$A \to B \textbf{ uno tres } B\ C \mid \textbf{dos } C \textbf{ tres } B\ C \mid \textbf{tres } B\ C \mid \textbf{cuatro} \mid \varepsilon$$

2. Reemplazar B dentro de $A \to B$ **uno tres** B C por todas sus producciones ($B \to A$ **cinco** C **seis**, $B \to \varepsilon$):

$$A \to A \textbf{ cinco } C \textbf{ seis uno tres } B\ C \mid \textbf{uno tres } B\ C$$

Ahora A aparece en primera posición, lo que constituye recursividad directa por izquierda.

2.3.2. Gramática con recursividad directa por izquierda

$$\begin{aligned}
S &\to B \textbf{ uno} \mid \textbf{dos } C \mid \varepsilon \\
A &\to A \textbf{ cinco } C \textbf{ seis uno tres } B\ C \\
&\mid \textbf{dos } C \textbf{ tres } B\ C \\
&\mid \textbf{uno tres } B\ C \\
&\mid \textbf{tres } B\ C \\
&\mid \textbf{cuatro} \mid \varepsilon \\
B &\to A \textbf{ cinco } C \textbf{ seis} \mid \varepsilon \\
C &\to \textbf{siete } B \mid \varepsilon
\end{aligned}$$

Se transforman las reglas de tipo $A \to A\ \alpha \mid \beta$ (donde β no empieza con A) en reglas recursivas por derecha: $A \to \beta\ A'$ y $A' \to \alpha\ A' \mid \varepsilon$.

1. Identificar la regla con recursividad directa por izquierda:

$$A \to A \textbf{ cinco } C \textbf{ seis uno tres } B\ C$$

2. Identificar α y β:

$$\alpha = \textbf{cinco } C \textbf{ seis uno tres } B\ C$$

$$\beta = \textbf{dos } C \textbf{ tres } B\ C \mid \textbf{uno tres } B\ C \mid \textbf{tres } B\ C \mid \textbf{cuatro} \mid \varepsilon$$

3. Aplicar la transformación $A \to \beta\ A'$ y $A' \to \alpha\ A' \mid \varepsilon$:

$$
\begin{aligned}
A &\to \textbf{dos}\ C\ \textbf{tres}\ B\ C\ A' \\
&\mid\ \textbf{uno tres}\ B\ C\ A' \\
&\mid\ \textbf{tres}\ B\ C\ A' \\
&\mid\ \textbf{cuatro}\ A' \\
&\mid\ A' \\
A' &\to \textbf{cinco}\ C\ \textbf{seis uno tres}\ B\ C\ A' \mid \varepsilon
\end{aligned}
$$

4. Como β incluye ε, la producción $A \to \varepsilon\ A'$ se simplifica a $A \to A'$.

2.3.3. Gramática sin recursividad por izquierda

$$
\begin{aligned}
S &\to B\ \textbf{uno} \mid \textbf{dos}\ C \mid \varepsilon \\
A &\to \textbf{dos}\ C\ \textbf{tres}\ B\ C\ A' \mid \textbf{uno tres}\ B\ C\ A' \mid \textbf{tres}\ B\ C\ A' \mid \textbf{cuatro}\ A' \mid A' \\
A' &\to \textbf{cinco}\ C\ \textbf{seis uno tres}\ B\ C\ A' \mid \varepsilon \\
B &\to A\ \textbf{cinco}\ C\ \textbf{seis} \mid \varepsilon \\
C &\to \textbf{siete}\ B \mid \varepsilon
\end{aligned}
$$

2.3.4. Implementación del ASDR

Se tomó como base el código `parser.py` del repositorio `https://github.com/Mariana909/ARBOL_SINTACTICO.git`, sobre el cual se realizaron las siguientes adaptaciones: se redefinieron los tokens para que correspondan a los terminales de cada gramática, se implementó una función por cada no terminal siguiendo la estructura de la gramática corregida, y se conservó el módulo de generación de imágenes que produce el árbol de sintaxis para cada cadena de entrada.

En el repositorio `https://github.com/Mariana909/LL1.git` se encuentra la implementación descrita, en el archivo `parser_gramatica2.py`. Para ejecutarla se usa el comando:

```
python3 parser_gramatica2.py entrada_g2.txt
```

donde `entrada_g2.txt` contiene las cadenas a analizar, una por línea. El programa imprime en consola si cada cadena es aceptada o rechazada, y genera una imagen del árbol de sintaxis por cada entrada.

Salidas:

```
ACEPTADA  "dos"
ACEPTADA  "uno"
ACEPTADA  "dos siete"
RECHAZADA  "cuatro"
RECHAZADA  "dos tres cuatro"
RECHAZADA  "dos siete cuatro cinco"
```

Figura 10: Salidas de parser_gramatica2.py con entrada_g2.txt

A continuación un ejemplo de cadena aceptada en la Figura 11 y una cadena rechazada en la Figura 12

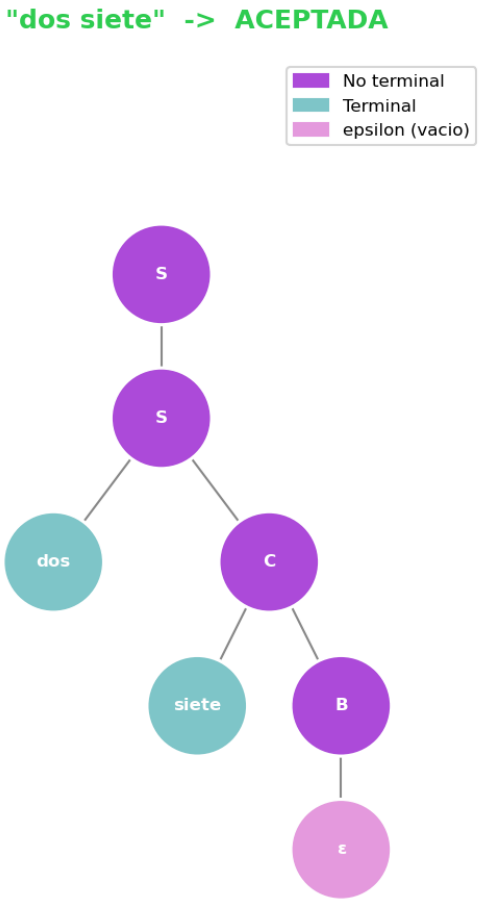

Figura 11: Árbol de sintaxis de la entrada `dos siete`

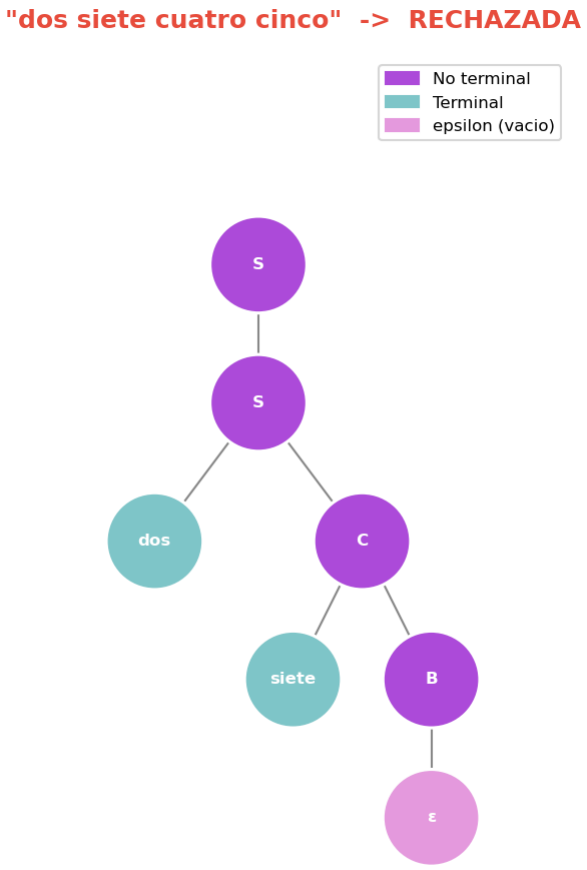

Figura 12: Árbol de sintaxis de la entrada `dos siete cuatro cinco`

3. Ejercicio 3

3.1. Descripción del problema

Eliminar recursión por izquierda , hallar conjunto de PRIMEROS, SIGUIENTES, PREDICCION, determinar si la gramática es LL(1) e implementar la función para cada no terminal en un ASDR.

Gramática:

$$
\begin{aligned}
S &\rightarrow A\ B\ C \\
S &\rightarrow S\ \textbf{uno} \\
A &\rightarrow \textbf{dos}\ B\ C \\
A &\rightarrow \varepsilon \\
B &\rightarrow C\ \textbf{tres} \\
B &\rightarrow \varepsilon \\
C &\rightarrow \textbf{cuatro}\ B \\
C &\rightarrow \varepsilon
\end{aligned}
$$

Figura 13: Gramática 3

3.2. Solución primeros cinco items

3.2.1. Eliminar recursión por izquierda

Se transforman las reglas de tipo $A \rightarrow A\ \alpha \mid \beta$ (donde β no empieza con A) en reglas recursivas por derecha: $A \rightarrow \beta\ A'$ y $A' \rightarrow \alpha\ A' \mid \varepsilon$.

1. Identificar la regla con recursividad directa por izquierda: $S \rightarrow S$ **uno**
2. Identificar α y β:
$$A = S, \quad \alpha = \textbf{uno}, \quad \beta = A\ B\ C$$
3. Aplicar la transformación:
$$S \rightarrow A\ B\ C\ S' \qquad S' \rightarrow \textbf{uno}\ S' \mid \varepsilon$$

3.2.2. Gramática corregida

$$
\begin{aligned}
S &\rightarrow A\ B\ C\ S' \\
S' &\rightarrow \textbf{uno}\ S' \mid \varepsilon \\
A &\rightarrow \textbf{dos}\ B\ C \mid \varepsilon \\
B &\rightarrow C\ \textbf{tres} \mid \varepsilon \\
C &\rightarrow \textbf{cuatro}\ B \mid \varepsilon
\end{aligned}
$$

3.2.3. Conjunto de PRIMEROS y SIGUIENTES

Se hizo uso del código psp.py del repositorio `https://github.com/Mariana909/PRIMEROS_SIGUIENTES_PREDICCION.git` con Gramatica3.txt como archivo de entrada, este archivo se encuentra en el repositorio `https://github.com/Mariana909/LL1.git`.

No Terminal	PRIMEROS	SIGUIENTES
S	{ _, cuatro, dos, tres, uno }	{ $ }
S'	{ _, uno }	{ $ }
A	{ _, dos }	{ $, cuatro, tres, uno }
B	{ _, cuatro, tres }	{ $, cuatro, tres, uno }
C	{ _, cuatro }	{ $, cuatro, tres, uno }

Figura 14: Conjnto de PRIMEROS y de SIGUIENTES de Gramatica3

3.2.4. Conjunto PREDICCION y LL(1)

Además de la tabla de PRIMEROS y SIGUIENTES, se obtuvo la tabla de PREDICCION a partir de la ejecución anterior.

Regla	Conjunto de Prediccion
S -> A B C S'	{ $, cuatro, dos, tres, uno }
S' -> uno S'	{ uno }
S' -> _	{ $ }
A -> dos B C	{ dos }
A -> _	{ $, cuatro, tres, uno }
B -> C tres	{ cuatro, tres }
B -> _	{ $, cuatro, tres, uno }
C -> cuatro B	{ cuatro }
C -> _	{ $, cuatro, tres, uno }

Figura 15: Conjunto de PREDICCION de Gramatica3

3.2.5. Condiciones para LL(1)

Se evalúa si la Gramática 3 cumple las condiciones para ser LL(1).

- **Sin recursividad por izquierda**: Se modificó la gramática para eliminar la recursividad por izquierda, por lo que se cumple esta condición.
- **Sin factores comunes por izquierda**: La gramática no cuenta con factores comunes por izquierda, por lo que se cumple esta condición.
- **Conjuntos de predicción disjuntos**: Una gramática es LL(1) si, para cada no-terminal con dos o más reglas, los conjuntos de predicción de dichas reglas son disjuntos. Revisando la tabla de predicción, se encuentran los siguientes conflictos:
  - $B \to C$ **tres** y $B \to \varepsilon$ comparten los símbolos **cuatro** y **tres**.
  - $C \to$ **cuatro** B y $C \to \varepsilon$ comparten el símbolo **cuatro**.

  Estos conflictos son inherentes a la gramática: surgen de la recursión mutua entre B y C, que provoca que sus conjuntos SIGUIENTES sean casi idénticos e inevitablemente se solapen con los conjuntos PRIMEROS de sus alternativas no vacías. No es posible resolverlos mediante factorización por izquierda.

Se concluye que la Gramática 3 **no es LL(1)**. El analizador sintáctico descendente recursivo se implementa igualmente, pero incorpora *backtracking*: cuando una alternativa falla, el parser retrocede la posición y prueba la siguiente. Esto implica que en el peor caso el análisis no es lineal.

3.3. Función para cada no terminal en un ASDR

Se tomó como base el código `parser.py` del repositorio `https://github.com/Mariana909/ARBOL_SINTACTICO.git`, sobre el cual se realizaron las siguientes adaptaciones: se redefinieron los tokens para que correspondan a los terminales de cada gramática, se implementó una función por cada no terminal siguiendo la estructura de la gramática corregida, y se conservó el módulo de generación de imágenes que produce el árbol de sintaxis para cada cadena de entrada.

En el repositorio `https://github.com/Mariana909/LL1.git` se encuentra la implementación descrita, en el archivo `parser_gramatica3.py`. Para ejecutarla se usa el comando:

```
python3 parser_gramatica3.py entrada_g3.txt
```

donde `entrada_g3.txt` contiene las cadenas a analizar, una por línea. El programa imprime en consola si cada cadena es aceptada o rechazada, y genera una imagen del árbol de sintaxis por cada entrada.

Salidas:

```
ACEPTADA  "dos tres"
ACEPTADA  "cuatro tres"
ACEPTADA  "dos cuatro tres uno"
RECHAZADA  "dos dos"
RECHAZADA  "uno dos"
RECHAZADA  "cuatro cuatro tres uno"
```

Figura 16: Salidas de parser_gramatica3.py con entrada_g3.txt

A continuación un ejemplo de cadena aceptada en la Figura 17 y una cadena rechazada en la Figura 18

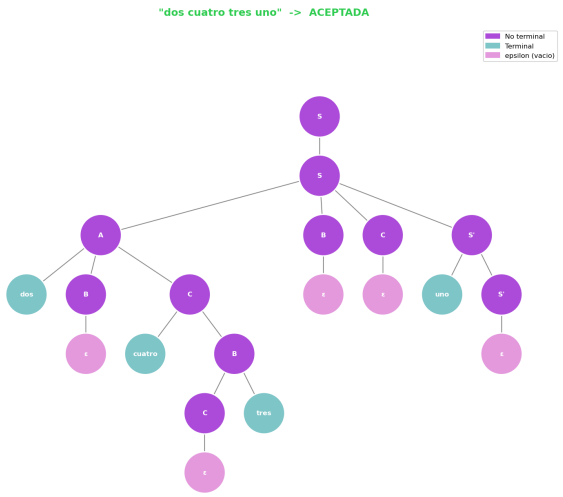

Figura 17: Árbol de sintaxis de la entrada `dos cuatro tres uno`

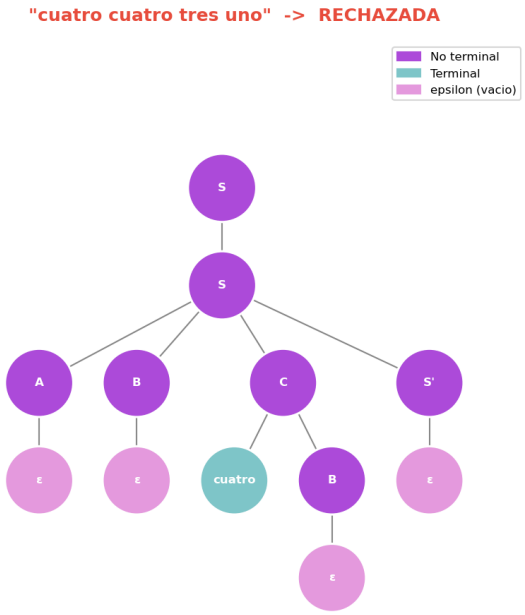

Figura 18: Árbol de sintaxis de la entrada `cuatro cuatro tres uno`